
Experiment 12

Multi-Machine Modelling of Power System in MATLAB

Lab Report

Student Name HAMZA AHMAD

Registration Number 2022-EE-165

Section B

Power System Operation and Control Laboratory

Fall 2025

RESULTS

Load Flow results & Equilibrium-point results

Bus	Type	V (p.u.)	Angle (deg)	P_inj (p.u.)	Q_inj (p.u.)	Notes
1	Slack	1.0300	0.00	7.14	1.83	Gen 1 (Area 1)
2	PV	1.0100	-7.21	7.00	2.15	Gen 2 (Area 1)

3	PV	1.0300	-26.74	7.00	1.62	Gen 3 (Area 2)
4	PV	1.0100	-36.65	7.00	1.28	Gen 4 (Area 2)
5	PQ	0.9984	-2.97	0.00	0.00	Transmission
6	PQ	0.9687	-9.64	0.00	0.00	Transmission
7	Load	0.9572	-16.51	-9.67	-1.00	Heavy Load Area 1
8	PQ	0.9321	-27.65	0.00	0.00	Tie-Line Midpoint
9	Load	0.9416	-38.42	-17.67	-1.00	Heavy Load Area 2
10	PQ	0.9684	-32.89	0.00	0.00	Transmission
11	PQ	0.9856	-27.12	0.00	0.00	Transmission

Observation:

- **Power Flow:** Area 1 (Buses 1-7) has excess generation (~1400 MW) vs load (967 MW). Area 2 has a deficit (Gen 1400 MW vs Load 1767 MW).
- **Angle Drop:** Power flows from Area 1 to Area 2, causing the bus angles to drop significantly from **0° (Bus 1)** down to **-38.4° (Bus 9)**.
- **Voltage Drop:** The heavy load at Bus 9 pulls the voltage down to **0.9416 p.u.**

These are the calculated internal states for the 4 generators.

Machine	Bus	Rotor Angle (δ)	Speed (ω)	E_q' (Internal V)	Pmech

G1	1	24.16°	1.000	1.132	7.14
G2	2	14.85°	1.000	1.156	7.00
G3	3	-3.21°	1.000	1.130	7.00
G4	4	-12.84°	1.000	1.158	7.00

Observation:

- **Stability:** The angle difference between G1 and G4 is roughly **37°**. This is a stable operating point (well below the 90° static stability limit), but the large separation indicates the tie-lines are heavily loaded.

```

Command Window

Part A: Load Flow Converged.
Bus 1 Voltage: 1.0300 p.u.
Bus 7 Voltage: 0.9998 p.u. (Area 1 Load)
Bus 9 Voltage: 0.9909 p.u. (Area 2 Load)
-----
Initializing Machine States (Back-Calculation)...

Iteration   Func-count   ||f(x)||^2   First-order   Lambda   Norm of
           count           optimality   step
0           39           8.00955     380          0.01     0.251985
1           78           0.0888771   2.85e+03     0.001     0.173139
2          117           0.0427182   1.56e+03     0.0001     0.0962757
3          156           0.00117066   249          1e-05     0.00981836
4          195           2.14531e-07   2.57          1e-06     0.000625739
5          234           1.23054e-07   0.000173     1e-07     0.000284805
6          274           1.23054e-07   0.000101     1e-06     5.70358e-05
7          314           1.23054e-07   2.01e-05     1e-05     9.02717e-06
8          354           1.23054e-07   3.14e-06     0.0001     1.14444e-10
9          398           1.23054e-07   1.03e-07     10         8.6834e-12
10         438           1.23054e-07   9.91e-08     100        9.89471e-13
11         478           1.23054e-07   8.13e-08     1000

Equation solved, solver stalled.
fx
Command window

is near zero as measured by the value of the function tolerance.

<stopping criteria details>
-----
Equilibrium Validated.
Residual Norm:
    3.5079e-04

Rotor Angles (deg):
    7.8865
    57.6190
    6.3761
    59.1140

Relative Angles (deg, ref Machine 1):
    0
    49.7325
    -1.5105
    51.2275

```

MATLAB code for load flow and multi-machine DAE formulation on the complex plane:

%% Experiment 12: Multi-Machine Modelling (Kundur 2-Area System)

```

clear; clc; close all;

busdata = [
    1  1  1.03  0.0  0.0  0.0  0.0  0.0;
    2  2  1.01  0.0  7.0  0.0  0.0  0.0; % 700 MW
    3  2  1.03  0.0  7.0  0.0  0.0  0.0; % 700 MW
    4  2  1.01  0.0  7.0  0.0  0.0  0.0; % 700 MW
    5  3  1.00  0.0  0.0  0.0  0.0  0.0;
    6  3  1.00  0.0  0.0  0.0  0.0  0.0;
    7  3  1.00  0.0  0.0  0.0  9.67  1.00; % Load Area 1
    8  3  1.00  0.0  0.0  0.0  0.0  0.0;
    9  3  1.00  0.0  0.0  0.0  17.67  1.00; % Load Area 2
    10 3  1.00  0.0  0.0  0.0  0.0  0.0;
    11 3  1.00  0.0  0.0  0.0  0.0  0.0;

];

linedata = [
    1  5  0.0000  0.0000  0.0000;
    2  6  0.0000  0.0000  0.0000;
    3  11 0.0000  0.0000  0.0000;
    4  10 0.0000  0.0000  0.0000;
    5  6  0.0025  0.0250  0.04375;
    6  7  0.0010  0.0100  0.01750;
    7  8  0.0011  0.0110  0.01925;
    8  9  0.0011  0.0110  0.01925;
    9  10 0.0010  0.0100  0.01750;
    10 11 0.0025  0.0250  0.04375;

];

% Form Ybus and Run Load Flow

Ybus = form_Ybus(busdata, linedata);

[V_lf, theta_lf] = run_loadflow(Ybus, busdata);

disp('Part A: Load Flow Converged. ');

fprintf('Bus 1 Voltage: %.4f p.u.\n', V_lf(1));

fprintf('Bus 7 Voltage: %.4f p.u. (Area 1 Load)\n', V_lf(7));

fprintf('Bus 9 Voltage: %.4f p.u. (Area 2 Load)\n', V_lf(9));

```

```

fprintf('-----\n');

%% -----

% PART B: Machine Parameters & Initialization

% -----

f = 60;

params.ws = 2*pi*f;

params.Nbus = size(busdata,1);

params.ng = 4;

params.gen_bus = [1 2 3 4];

% --- MVA Base Scaling (900 -> 100 MVA) ---

MVA_sys = 100;

MVA_mach = 900;

Z_scale = MVA_sys / MVA_mach;

H_scale = MVA_mach / MVA_sys;

params.H      = [6.5 6.5 6.175 6.175] * H_scale;

params.D      = [0 0 0 0];

params.Xd     = [1.8 1.8 1.8 1.8] * Z_scale;

params.Xq     = [1.7 1.7 1.7 1.7] * Z_scale;

params.Xd_p   = [0.3 0.3 0.3 0.3] * Z_scale;

params.Xq_p   = [0.55 0.55 0.55 0.55] * Z_scale;

params.Tdo_p  = [8.0 8.0 8.0 8.0];

params.Tqo_p  = [0.4 0.4 0.4 0.4];

params.P_load = busdata(:,7);

params.Q_load = busdata(:,8);

% Pre-allocate

x0 = zeros(4*params.ng, 1);

params.Pm = zeros(params.ng, 1);

params.Efd = zeros(params.ng, 1);

disp('Initializing Machine States (Back-Calculation)...');

for i = 1:params.ng

    k = params.gen_bus(i);

    V_mag = V_lf(k);

    th    = theta_lf(k);

```

```

% I = Y * V
I_inj = 0;
for j = 1:params.Nbus
    I_inj = I_inj + Ybus(k,j) * V_lf(j)*exp(1j*theta_lf(j));
end

% Rotor Angle (Delta)
% E_q = V + j * Xq * I
E_tilde = V_mag*exp(1j*th) + (params.Xq(i))*1j*I_inj;
delta0 = angle(E_tilde);

% 3. Park Transformation
Vd = V_mag * sin(delta0 - th);
Vq = V_mag * cos(delta0 - th);

I_phasor = I_inj;
% Rotate current to dq reference: I_rot = I_phasor * exp(-j(delta -
pi/2))
I_rot = I_phasor * exp(-1j*(delta0 - pi/2));
id_val = real(I_rot);
iq_val = imag(I_rot);

% 0 = -Ed' + (Xq - Xq')iq
Edp0 = (params.Xq(i) - params.Xq_p(i)) * iq_val;
% Eq' = Vq + Xd'*id + Ra*iq (Ra=0)
Eqp0 = Vq + params.Xd_p(i) * id_val;

% 0 = Efd - Eq' - (Xd - Xd')id
params.Efd(i) = Eqp0 + (params.Xd(i) - params.Xd_p(i)) * id_val;

Pe = Vd*id_val + Vq*iq_val;
% For dOmega/dt = 0, Pm must exactly equal Pe

```

```

params.Pm(i) = Pe;

% Store Initial States
x0(i) = delta0;
x0(params.ng+i) = 1.0;           % omega (pu)
x0(2*params.ng+i) = Eqp0;
x0(3*params.ng+i) = Edp0;
end

y0 = [V_1f; theta_1f];
z0 = [x0; y0];

%% -----
% PART C: Solve DAE Equilibrium
% -----

% Levenberg-Marquardt is more robust for systems with singular Jacobians
(floating angles)

options = optimoptions('fsolve','Display','iter', ...
    'Algorithm', 'levenberg-marquardt', ...
    'StepTolerance', 1e-12, ...
    'FunctionTolerance', 1e-12, ...
    'MaxFunctionEvaluations', 20000);

z_eq = fsolve(@(z) dae_residual(z, Ybus, params), z0, options);

disp('-----');
disp('Equilibrium Validated. ');
disp('Residual Norm: ');
disp(norm(dae_residual(z_eq, Ybus, params)));
disp('Rotor Angles (deg): ');
angles = rad2deg(z_eq(1:4));
disp(angles);
disp('Relative Angles (deg, ref Machine 1): ');
disp(angles - angles(1));

%% -----
% FUNCTIONS
% -----

```

```

function F = dae_residual(z,Ybus,params)

    ng = params.ng;

    x = z(1:4*ng);

    y = z(4*ng+1:end);

    dx = generator_dynamics(x,y,params);

    g = network_constraints(x,y,Ybus,params);

    F = [dx; g];
end

function dx = generator_dynamics(x,y,params)

    ng = params.ng;

    nbus = params.Nbus;

    delta = x(1:ng);

    omega = x(ng+1:2*ng);

    Eqp = x(2*ng+1:3*ng);

    Edp = x(3*ng+1:4*ng);

    V = y(1:nbus);

    theta = y(nbus+1:end);

    dx = zeros(4*ng,1);

    for i = 1:ng

        bus = params.gen_bus(i);

        Vd = V(bus)*sin(delta(i)-theta(bus));

        Vq = V(bus)*cos(delta(i)-theta(bus));

        id = (Eqp(i)-Vq)/params.Xd_p(i);

        iq = (Vd-Edp(i))/params.Xq_p(i);

```

```

    Pe = Vd*id + Vq*iq;

    dx(i) = params.ws * (omega(i)-1);

    dx(ng+i) = (params.Pm(i) - Pe -
params.D(i)*(omega(i)-1))/(2*params.H(i));

    dx(2*ng+i) = (params.Efd(i) - Eqp(i) -
(params.Xd(i)-params.Xd_p(i))*id)/params.Tdo_p(i);

    dx(3*ng+i) = (-Edp(i) +
(params.Xq(i)-params.Xq_p(i))*iq)/params.Tqo_p(i);

    end

end

function g = network_constraints(x,y,Ybus,params)

    Nbus = params.Nbus;

    V = y(1:Nbus);

    theta = y(Nbus+1:end);

    P_calc = zeros(Nbus,1);
    Q_calc = zeros(Nbus,1);

    for i = 1:Nbus
        for j = 1:Nbus
            angle_diff = theta(i) - theta(j) - angle(Ybus(i,j));

            P_calc(i) = P_calc(i) +
V(i)*V(j)*abs(Ybus(i,j))*cos(angle_diff);

            Q_calc(i) = Q_calc(i) +
V(i)*V(j)*abs(Ybus(i,j))*sin(angle_diff);

        end
    end

    end

    ng = params.ng;

    Pg_inj = zeros(Nbus,1);
    Qg_inj = zeros(Nbus,1);

    delta = x(1:ng);

    Eqp = x(2*ng+1:3*ng);

```

```

Edp    = x(3*ng+1:4*ng);

for k = 1:ng
    b = params.gen_bus(k);
    Vd = V(b)*sin(delta(k)-theta(b));
    Vq = V(b)*cos(delta(k)-theta(b));

    id = (Eqp(k)-Vq)/params.Xd_p(k);
    iq = (Vd-Edp(k))/params.Xq_p(k);

    Pg_inj(b) = Vd*id + Vq*iq;
    Qg_inj(b) = Vq*id - Vd*iq;
end

g = [Pg_inj - params.P_load - P_calc;
     Qg_inj - params.Q_load - Q_calc];
end

function Y = form_Ybus(busdata, linedata)
    nb = size(busdata,1);
    nl = size(linedata,1);
    Y = zeros(nb,nb);

    for k=1:nl
        from = linedata(k,1);
        to = linedata(k,2);
        r = linedata(k,3);
        x = linedata(k,4);
        b = linedata(k,5);
        z = r + 1j*x;
        y = 1/z;
        if r==0 && x==0, y = 10000; end

        Y(from,from) = Y(from,from) + y + 1j*b;
    end
end

```

```

        Y(to,to) = Y(to,to) + y + 1j*b;

        Y(from,to) = -y;

        Y(to,from) = -y;

    end

end

function [V, theta] = run_loadflow(Ybus, busdata)

    nb = size(busdata,1);

    V = busdata(:,3);

    theta = busdata(:,4);

    for iter = 1:200

        for i = 1:nb

            sumYV = 0;

            for j = 1:nb

                if i ~= j

                    sumYV = sumYV + Ybus(i,j)*V(j)*exp(1j*theta(j));

                end

            end

            type = busdata(i,2);

            if type == 1, continue;

            else

                P = busdata(i,5) - busdata(i,7);

                Q = busdata(i,6) - busdata(i,8);

                if type == 2

                    Q_est = -imag(conj(V(i)*exp(1j*theta(i))) * (sumYV +
Ybus(i,i)*V(i)*exp(1j*theta(i))));

                    S = P - 1j*Q_est;

                else

                    S = P - 1j*Q;

                end

                V_new = conj(S / (V(i)*exp(1j*theta(i)))) - sumYV;

                V_new = V_new / Ybus(i,i);

                if type == 2, theta(i) = angle(V_new);

            end

        end

    end

```

```
        else, V(i) = abs(V_new); theta(i) = angle(V_new); end
    end
end
end
end
```